¿Humano o máquina? Detección distribuida de texto generado por IA con Spark MLlib

Alicia Losada Sánchez, Carlos Francisco Orejuela Lluncor, and Hugo Varela Sánchez

Aprendizaje Automático a Gran Escala, Universidade da Coruña

Resumen Clasificamos 1,39 millones de textos en inglés como humanos o generados por IA con Spark MLlib. Antes de entrenar eliminamos los atajos que deja la construcción del dataset (longitud y marcas de formato propias de cada fuente). Comparamos cuatro familias de features (léxicas, de caracteres, estilométricas y de previsibilidad según un modelo de n-gramas), varios clasificadores y su combinación por stacking. El mejor modelo individual, una regresión logística sobre unigramas y bigramas, alcanza una AUC-PR de 0,837 en test frente a una línea base de 0,225. El stacking sube a 0,863, pero el análisis por fuente muestra que parte del acierto viene del género del texto y no de la autoría.

Palabras clave: Detección de texto generado · Spark MLlib · Estilometría · Stacking

1. Introducción

Los modelos de lenguaje producen texto difícil de distinguir del humano, lo que afecta a la evaluación académica, a la moderación de contenidos y a la fiabilidad de lo que se publica en la web. Los detectores más precisos usan otro modelo de lenguaje grande para medir la probabilidad del texto [2,4], lo que es caro de aplicar a millones de documentos. Nos preguntamos hasta dónde llegan features clásicas, baratas y calculables en paralelo con Spark [6,3].

Usamos *AI Text Detection Pile*: 1.392.522 textos (1,9 GB en 7 ficheros Parquet), un 73,8 % humanos y un 26,2 % generados por IA. El preprocesado, las features y la búsqueda de hiperparámetros no caben cómodamente en una máquina, así que se ejecutó en el CESGA.

2. Datos y preprocesado

Fuentes. El dataset solo indica si cada texto es humano o IA, pero los `id` están ordenados por origen. Analizando la longitud y las marcas de formato a lo largo del `id` identificamos ocho bloques que coinciden en tamaño con lo que declara la ficha: ensayos de IvyPanda, WebText, respuestas humanas de HC3 y relatos de Reddit WritingPrompt en el lado humano; GPT-3, GPT-2, GPT-J y ChatGPT en el de la IA. Esta columna (`subsource`) es inferida y solo se usa para analizar resultados, nunca como feature, porque el `id` separa las clases al 100 %.

Atajos. Cada fuente deja huellas que no tienen que ver con quién escribe. La más fuerte es la longitud: la mediana es de 305 palabras en la IA y 190 en los humanos, porque los relatos de Reddit vienen recortados a unos 1.000 caracteres y GPT-2 llega a su límite de generación. También hay marcas de formato casi exclusivas de una clase: puntuación separada por espacios en el 85 % de HC3, o un espacio inicial en el 98 % de GPT-3.

Limpieza. Aplicamos las mismas reglas a las dos clases, sin mirar la etiqueta y solo con funciones nativas de Spark (sin UDF de Python): comillas normalizadas, títulos y asteriscos eliminados, puntuación separada unida de nuevo y espacios normalizados. Además descartamos siempre la primera frase, para que todos los textos arranquen a mitad de documento como los de Reddit, a costa de perder aperturas que sí son estilo (el "It's important to understand..." de ChatGPT). Tras la limpieza todas las marcas medidas bajan a cerca del 0 %.

Ventana y duplicados. Cada texto se recorta a sus primeras 100 palabras y se descartan los más cortos. Así se conserva Reddit (trozos de unas 170 palabras) con texto suficiente para la estilometría. Quedan unos 1,19 millones de textos con un 22 % de IA, aunque las fuentes de respuestas cortas pierden mucho (sobrevive el 14,5 % de GPT-3 y el 25 % de HC3). Después eliminamos los duplicados exactos, conservando la copia de menor `id`, y los textos que aparecen con las dos etiquetas.

3. Features

1. **Léxicas.** Unigramas y bigramas de palabras en minúscula, con y sin *stopwords*, contados con `HashingTF` (2^{18} posiciones).
2. **Caracteres.** N-gramas de 3 a 5 caracteres, conservando mayúsculas, espacios y puntuación. MLlib no tiene n-gramas de caracteres, así que los construimos con `RegexTokenizer` (un token por carácter) seguido de `NGram`.
3. **Estilométricas.** 32 medidas por texto: riqueza léxica (TTR, hápax), histograma y media de la longitud de palabra, número y longitud de las frases y su variabilidad, frecuencia de ocho signos de puntuación, mayúsculas, dígitos, contracciones, repetición de bigramas y trigramas y proporción de *stopwords*.
4. **Previsibilidad.** Un modelo de trigramas con *Stupid Backoff* [1] entrenado con los textos humanos de `lm_fit`. Lo elegimos porque no normaliza: evita los pesos de Katz o Kneser-Ney, que exigen agregaciones extra, y sus autores lo diseñaron para corpus a escala web. Los conteos son un *word count* distribuido. La primera versión lo hacía con RDDs (`flatMap` y `reduceByKey`); la sustituimos por `explode` y `groupBy` sobre DataFrames, porque el RDD serializa cada n-grama entre la JVM y Python, mientras que con DataFrames todo el conteo se queda en la JVM. Para puntuar cada palabra, las tablas de n-gramas se cruzan con los textos mediante *broadcast joins* sobre claves hash. De cada texto salen 9 features: media, desviación y percentil 10 del log-score, nivel del backoff usado y, siguiendo a GLTR [2], la fracción de palabras que estaban entre las 1, 10 y 100 continuaciones más probables.

4. Modelos y metodología

Sobre las features dispersas (familias 1 y 2) entrenamos Naive Bayes multinomial con los conteos, y regresión logística y `LinearSVC` con TF-IDF. Sobre las densas (3 y 4) usamos regresión logística con `StandardScaler`, GBT y Random Forest. La partición es 70/15/15 (train, val, test). De train se aparta además el 30 % de los textos humanos (`lm_fit`), que solo se usa para el modelo de n-gramas; por eso train queda con un 29 % de IA.

Los hiperparámetros se eligen con `CrossValidator` de 3 folds sobre una muestra estratificada del 15 % de train, con folds también estratificados. Las rejillas son pequeñas (de 3 a 6 combinaciones) porque cada una cuesta tres ajustes. Con la mejor combinación se reentrena el `Pipeline` con todo train. Resultaron elegidos: LR léxica $\lambda = 0{,}01$ (L2), SVC $\lambda = 0{,}01$, NB $\alpha = 0{,}1$, GBT de estilo con profundidad 5 y 100 iteraciones, GBT de previsibilidad con profundidad 3, RF con 100 árboles y profundidad 12, y LR densas sin regularización.

Métricas. La métrica principal es la AUC-PR de la clase IA. Con un 22,5 % de positivos la AUC-ROC es optimista, porque los muchos verdaderos negativos la inflan, mientras que la AUC-PR se centra en la clase que nos interesa [5]. Damos además la AUC-ROC y la tasa de detección con un 1 % de falsos positivos, que es lo relevante si acusar a un humano tiene un coste alto. Para las métricas con umbral (precisión, recall, F1, FPR) el umbral se fija en val maximizando el F1 y se aplica sin tocar a test. Por último, calculamos el recall en cada generador y el FPR en cada fuente humana.

Stacking. Tomamos el mejor modelo de cada familia según su AUC-PR en val. Un meta-modelo (escalado y regresión logística) aprende a combinar sus scores en val, donde los modelos base no han visto los textos; en train sus scores serían optimistas. La ablación reentrena el meta-modelo sin cada familia.

5. Organización del código

`src/main.py` ejecuta todo de principio a fin: `clean_data.py` (limpieza, ventana y particiones), un módulo por familia en `features_families/`, `models.py` (pipelines y rejillas), `train.py`, `evaluate.py` y `stacking.py`; los parámetros comunes están en `config.py`. Cada paso escribe en disco y se salta si su salida está completa, lo que permite reanudar en el CESGA, y el Parquet limpio está particionado por `split` para leer solo lo necesario.

6. Resultados

La Tabla 1 resume los resultados. Las features léxicas son las mejores, y dentro de ellas la regresión logística. Quitar las *stopwords* empeora tanto el SVC (de 0,789 a 0,696) como Naive Bayes (de 0,692 a 0,648): buena parte de la señal está en cómo se usan las palabras funcionales, no en el vocabulario temático. En

Tabla 1. Resultados en test (177.060 textos, 22,5 % de IA). La línea base de la AUC-PR es 0,225. TPR@1 %: detección de IA con un 1 % de humanos marcados.

Features	Modelo	AUC-PR	AUC-ROC	F1	FPR	TPR@1 %
Léxicas	LR	0,837	0,946	0,773	0,086	0,365
Léxicas	SVC	0,789	0,923	0,731	0,099	0,300
Léxicas	NB	0,692	0,889	0,651	0,149	0,197
Léxicas sin *stopwords*	SVC	0,696	0,883	0,660	0,144	0,194
Léxicas sin *stopwords*	LR	0,672	0,887	0,655	0,163	0,154
Léxicas sin *stopwords*	NB	0,648	0,883	0,644	0,177	0,130
Caracteres	NB	0,594	0,855	0,587	0,274	0,115
Estilo	GBT	0,645	0,871	0,629	0,180	0,144
Estilo	RF	0,634	0,864	0,619	0,173	0,137
Estilo	LR	0,531	0,818	0,567	0,225	0,067
Previsibilidad	GBT	0,369	0,668	0,423	0,438	0,046
Previsibilidad	RF	0,368	0,668	0,423	0,424	0,046
Previsibilidad	LR	0,333	0,625	0,396	0,454	0,037
Stacking	LR	**0,863**	**0,957**	**0,796**	**0,080**	**0,408**

estilo, GBT supera claramente a la regresión logística, así que las interacciones entre features importan. La previsibilidad es la familia más débil: el modelo de trigramas, entrenado sobre todo con relatos de Reddit, mide más el parecido con ese género que la previsibilidad del texto. Aun así, su regresión logística detecta el 82 % de GPT-3, frente al 32 % de la de estilo, lo que apunta a información complementaria (aunque en test solo hay 722 textos de GPT-3).

Val y test difieren menos de 0,01 en AUC-PR en todos los modelos: el umbral elegido en val se transfiere bien.

Tabla 2. Stacking en test. Izquierda: peso de cada familia (scores estandarizados) y caída de AUC-PR sin ella. Derecha: FPR por fuente humana y recall por generador.

Familia (modelo)	Peso	ΔAUC-PR
Léxicas (LR)	2,34	−0,112
Estilo (GBT)	0,88	−0,014
Previsibilidad (GBT)	0,41	−0,007
Caracteres (NB)	0,43	−0,001

Humanos	FPR	IA	Recall
Reddit	0,008	GPT-2	0,835
IvyPanda	0,063	GPT-3	0,819
WebText	0,275	GPT-J	0,913
HC3	0,297	ChatGPT	0,915

El stacking sube la AUC-PR a 0,863 (Tabla 2). Casi todo lo aporta la familia léxica; estilo y previsibilidad suman poco pero algo, y caracteres es prescindible. El análisis por fuente matiza el resultado global: el FPR del 8 % es una media dominada por Reddit (0,8 %), mientras que en WebText y HC3, prosa del mismo registro que la IA, más de uno de cada cuatro humanos se marca como IA. Es el caso más difícil y también el más realista.

7. Conclusiones

Con features clásicas y Spark MLlib se detecta texto generado con una AUC-PR de 0,837, muy por encima de la línea base, en un pipeline que procesa 1,4 millones de textos sin salir de la JVM. Para llegar a un resultado creíble fue tan importante limpiar los atajos del dataset como elegir el modelo, y el análisis por fuente muestra que aún queda uno, el género del texto.

Limitaciones. Las fuentes son inferidas a partir del `id`, y el límite entre IvyPanda y WebText mezcla unos 500 ensayos. Descartar la primera frase elimina algo de señal real. Casi todos los hiperparámetros elegidos están en el borde de la rejilla (α de Naive Bayes, λ de LR y SVC, número de árboles), así que conviene ampliarla.

Trabajo futuro. Evaluar dejando fuera un generador o un dominio entero, para medir la generalización a modelos no vistos; equilibrar los géneros entre clases; y cambiar los trigramas por la perplejidad de un modelo neuronal pequeño.

Referencias

1. Brants, T., Popat, A.C., Xu, P., Och, F.J., Dean, J.: Large language models in machine translation. In: Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL). pp. 858–867 (2007)
2. Gehrmann, S., Strobelt, H., Rush, A.M.: GLTR: Statistical detection and visualization of generated text. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations. pp. 111–116 (2019)
3. Meng, X., Bradley, J., Yavuz, B., et al.: MLlib: Machine learning in apache Spark. Journal of Machine Learning Research **17**(34), 1–7 (2016)
4. Mitchell, E., Lee, Y., Khazatsky, A., Manning, C.D., Finn, C.: DetectGPT: Zero-shot machine-generated text detection using probability curvature. In: Proceedings of the 40th International Conference on Machine Learning. PMLR, vol. 202, pp. 24950–24962 (2023)
5. Saito, T., Rehmsmeier, M.: The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLoS ONE **10**(3), e0118432 (2015)
6. Zaharia, M., Xin, R.S., Wendell, P., et al.: Apache Spark: A unified engine for big data processing. Communications of the ACM **59**(11), 56–65 (2016)